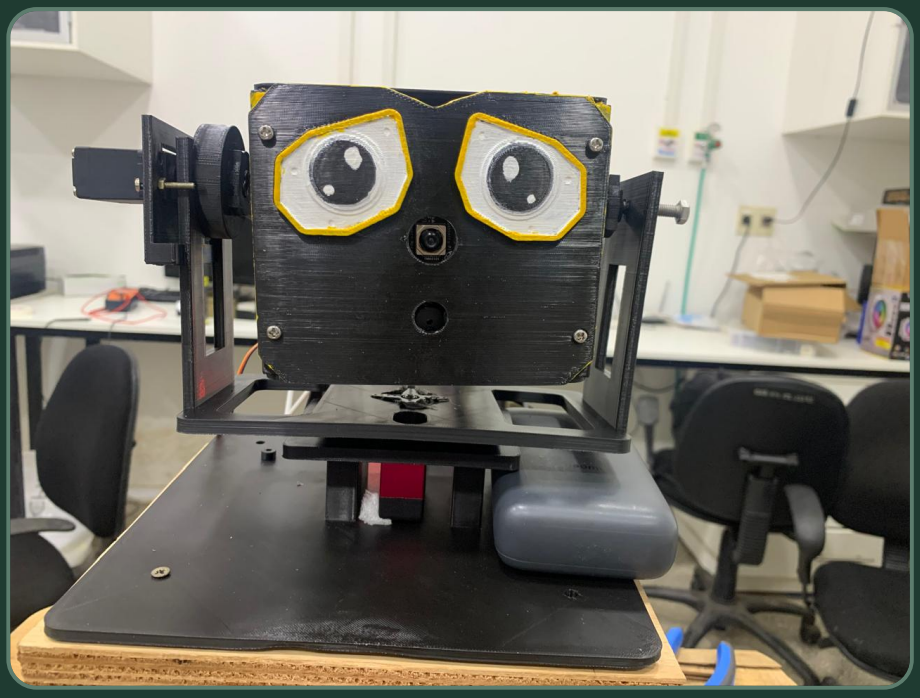
ROBÓTICA II · FIUNA · MTP ROBÓTICA SOCIAL · 2026

Wall-E

Proyecto Keepon pro

Guía completa del proyecto: construcción, hardware, control de los motores, software, decisiones y resultados



Cabeza social de 3 grados de libertad (yaw, pitch, roll) con visión artificial · Raspberry Pi 5 + ESP32-C3 ·
Integrantes: Mathias Aguilar · Héctor Velázquez · Pablo Otazú · Pedro Vera
Repositorio: github.com/HecVelaz/keepon-pro (versión **v1.0-entrega**) · Video: youtu.be/raOJljv1mVU
Guía actualizada el 08/11/2026 a partir del código y de la documentación del repo (`docs/`).

Índice

1 Qué es Wall-E y qué hace
2 Cómo avanzó el proyecto (línea de tiempo)
3 Construcción mecánica: del v2 al v6 R4
4 Hardware y electrónica: componentes, conexiones, pines y alimentación
5 Los servos: caracterización, calibración y conversión grados → pulso
6 Control de los motores: trayectorias, límites, watchdog y STOP
7 Firmware de la ESP32-C3 y protocolo serie
8 Arquitectura del software en la Raspberry Pi
9 Percepción: personas, rostros, mirada y pose
10 Eventos: definición operativa de cada uno
11 Detector de camisetas Olimpia / Cerro
12 Control de la mirada y foco de atención
13 Comportamientos y árbitro
14 Modos de personalidad (diagramas de estados)
15 Buzzer y sonidos
16 Monitoreo, log y operación (panel, comandos)
17 Por qué se decidió cada cosa
18 Resultados medidos, pruebas y lo que falta
19 **Para la defensa: preguntas probables y respuestas cortas**

> **Cómo leer esta guía**
> Cada capítulo explica *qué* hay, *cómo* funciona y *por qué* se eligió así. Los nombres en `monoespaciado` son archivos o parámetros del repositorio: todos los números (límites, umbrales, tiempos) salen de `config.yaml`, que concentra **todos** los parámetros. Los recuadros azules explican decisiones; los naranjas, advertencias o cosas sin probar.

1 Qué es Wall-E y qué hace

Wall-E (proyecto Keepon pro) es una cabeza robótica con ojos sobre un cuello de 3 grados de libertad. Una cámara en la cara ve a las personas; la Raspberry Pi decide qué hacer y una ESP32-C3 mueve los servos y toca sonidos en un buzzer. Mirar a alguien = centrarlo en la imagen.

3 GDL
yaw ±70° · pitch −20/+30° · roll ±20°

12–17 Hz
detección de personas en la Pi (pedido: ≥ 10 Hz)

36–39 Hz
lazo de control y árbitro

258
tests automáticos (36+ comparan firmware y simulador)

Qué percibe y cómo responde

Situación	Evento	Respuesta del robot
No hay nadie	—	**Observador**: centra la vista y explora la escena con barridos lentos de ±45°, sacadas rápidas a puntos de interés y pequeñas inclinaciones de roll.
Aparece una persona	PERSONA	**Seguimiento**: centra su rostro con yaw y pitch. Con varias personas prioriza a quien lo mira.
La persona saluda con la mano	SALUDO	3 cabeceos + sonido tipo R2D2.
Lleva la camiseta de Olimpia	CAMISETA_OLIMPIA	Silba la polca de Olimpia **sin dejar de seguirla**, una vez por aparición de la persona.
Lleva la camiseta de Cerro	CAMISETA_CERRO	Abucheo (silbido de reprobación).
La persona lo mira	ME_MIRA	Según el modo: Tímido aparta la mirada; Atrevido levanta el mentón.
La persona se va del campo visual	SALIENDO	En modo Maleducado: silbido de piropo, una vez por salida.
Se queda quieta 3 s	—	**Curiosidad** (extra): inclina la cabeza 15° en roll.
El operador manda un comando	OPERADOR	Desde el panel del celular: STOP y HOME (enclavados hasta RESUME), demostración de los 3 ejes, cambio de modo con beep, sonidos y volumen.

El sistema completo de un vistazo

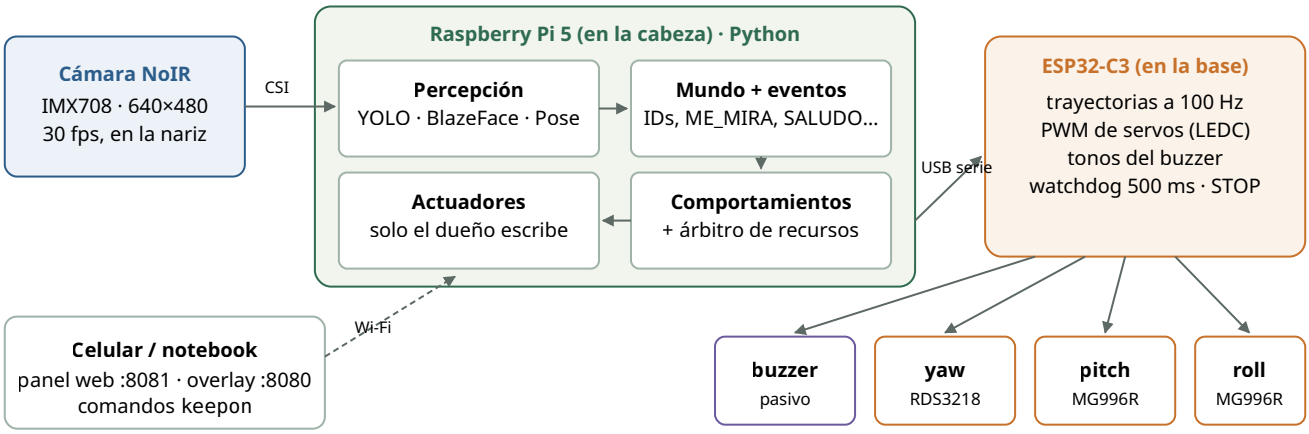


Figura 1. La Pi hace toda la inteligencia; la ESP32 solo ejecuta movimientos suaves y sonidos, de forma segura y sin bloquear.

2 Cómo avanzó el proyecto

De medir si la Pi alcanzaba, a un robot armado que sigue, saluda y reacciona a camisetas. La historia completa está en `docs/BITACORA.md`.

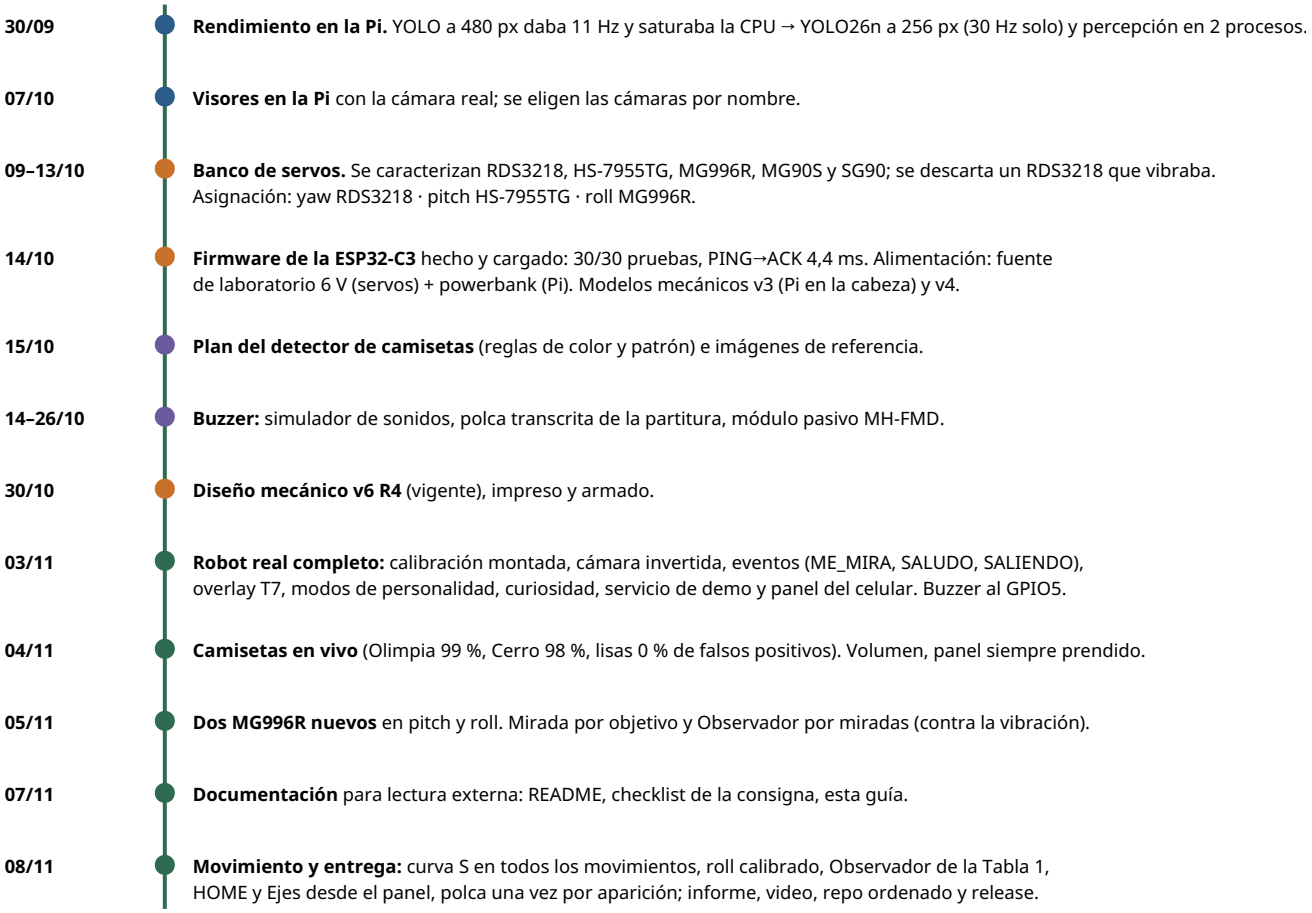
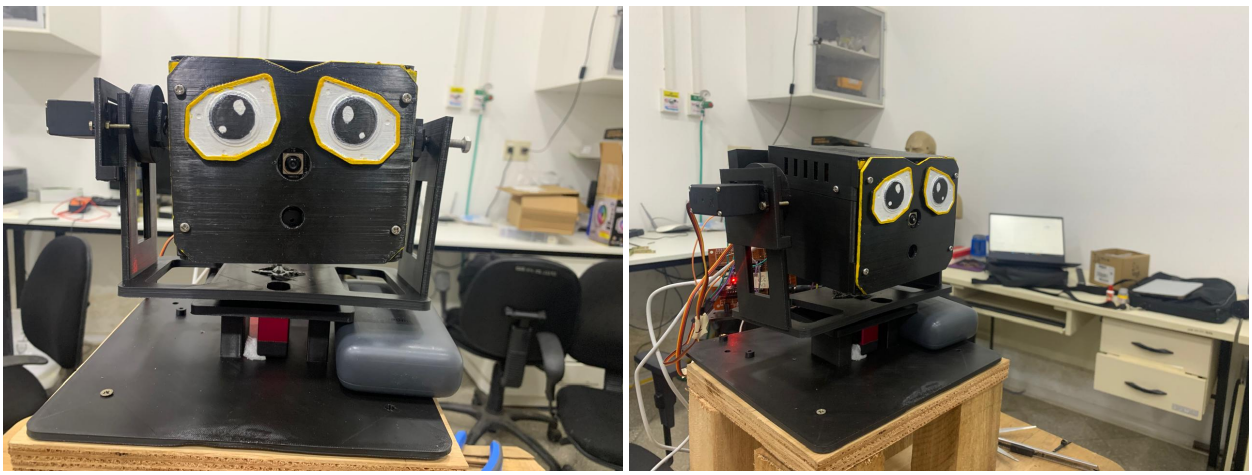


Figura 2. Azul: rendimiento · naranja: mecánica y motores · lila: sonido y camisetas · verde: integración en el robot.

> **Método de trabajo**
> Primero se escribió el robot completo en Python con un **ESP32 simulado** (`keepon/sim_esp32.py`) y personas sintéticas (`keepon/synthetic.py`), para probar el árbitro y los comportamientos sin mecánica. El firmware se escribió después como copia de ese simulador, y los tests comparan los dos paso a paso. Así cada pieza llegó al robot ya probada.

3 Construcción mecánica

Estructura impresa en 3D, diseñada por el grupo (CadQuery/Python + SolidWorks). El modelo vigente es el **v6 R4** (`docs/diseno_v6/`), armado desde el 03/11.



Wall-E armado: de frente (los ojos, la cámara como nariz y el buzzer debajo) y de costado (servo de pitch en la horquilla y la ESP32 en la base).

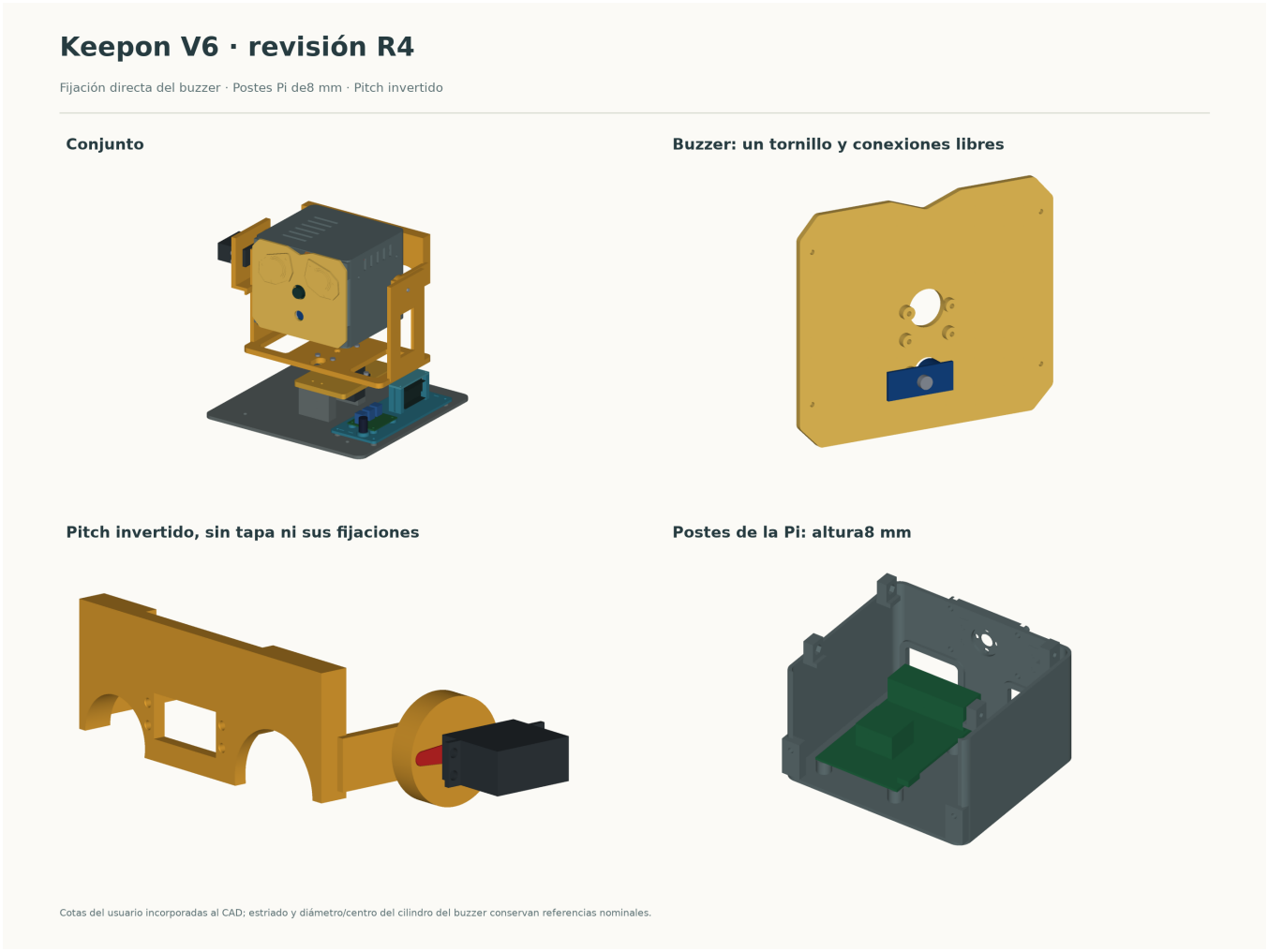
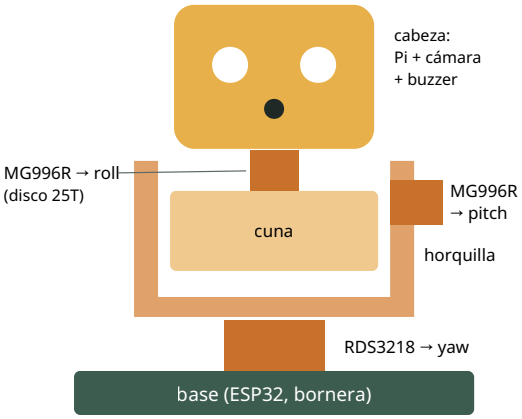


Figura 3. Modelo CAD v6 R4: conjunto (arriba a la izquierda), cara con el buzzer, cuna de pitch con el servo y caja de la cabeza con la Pi.

Cadena cinemática: qué mueve a qué



Eje	Servo	Va en	Mueve	Recorrido
yaw (guiñada)	RDS3218	la base	la horquilla	±70°
pitch (cabeceo)	MG996R	la horquilla	la cuna	−20° / +30°
roll (alabeo)	MG996R	la cuna	la cabeza (disco de 25T)	±20°

Los ejes de pitch y roll se cruzan en (0, 0, 163) mm y el centro óptico de la cámara está a la misma altura: al girar, la cámara rota sobre sí misma y no se desplaza, lo que simplifica el control de la mirada.

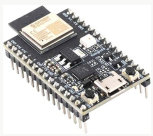
Evolución del diseño

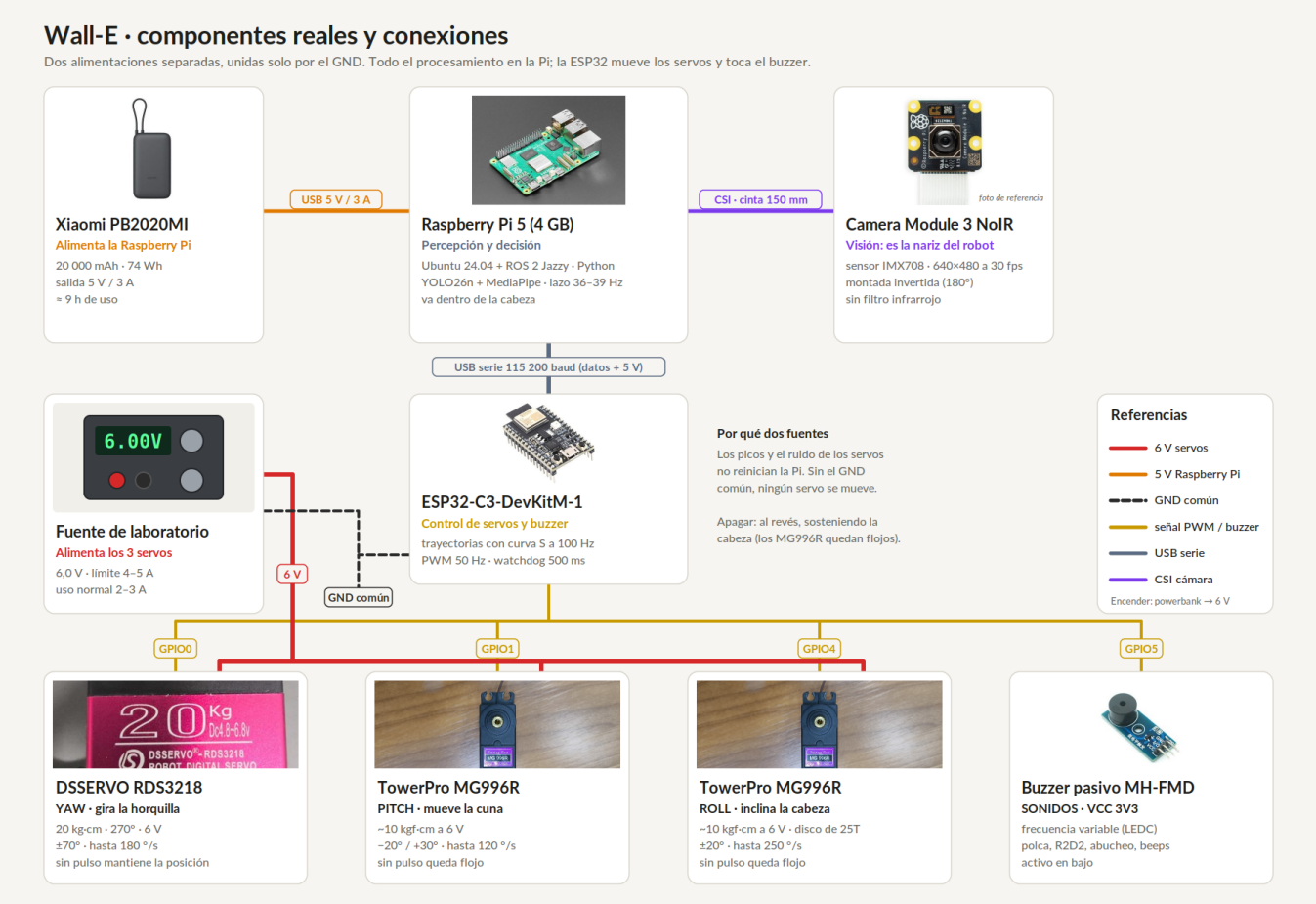
Versión	Fecha	Qué cambió y por qué
v2	oct.	Primer cuello completo de 3 ejes; el cuello se reutiliza en la v3.
v3	14/10	**Pi dentro de la cabeza** y la cámara como nariz: la cinta de la cámara mide ~150 mm y no llega a la base. Consecuencias: Pi alimentada por el GPIO si el USB-C queda tapado, y pitch con +30° arriba y −20° abajo (más abajo la cara toca el plato).
v4	14/10	Disposición con el eje de pitch sobre el de yaw y base en U (el vuelco sobre el yaw baja a 0,81 kgf·cm); caja base con powerbank, bornes de la fuente, interruptor y bornera.
v6 R4	30/10	Vigente y armado. Disco rojo de 25T para el roll, postes de la Pi de 8 mm, brazo de pitch invertido, buzzer fijado detrás de la cara con un tornillo. Verificado sin interferencias en 45 poses nominales.

> **Sin rodamiento de yaw**
> No se consiguió un rodamiento: el plato se apoya solo en el horn del RDS3218 (≈ 330 g, ≈ 1,3 kgf·cm de vuelco). Por eso importa que el peso quede centrado sobre el eje de yaw.

4 Hardware y electrónica

Componentes

	Componente	Función y datos clave
	**Raspberry Pi 5**, 4 GB	Toda la percepción, la decisión y el control. Ubuntu 24.04 + ROS 2 Jazzy (no se puede cambiar el SO: la Pi se usa en otros proyectos). Va en la cabeza.
	**Camera Module 3 NoIR** (IMX708)	640×480 a 30 fps. Montada invertida (`orientation_deg: 180`). Campo útil ≈ 37° × 28°: a 640×480, `camera_ros` recorta el sensor. Sin filtro IR: los colores se corren.
	**ESP32-C3-DevKitM-1** (clon CH340)	Pulsos de los servos y tonos del buzzer. Conectada a la Pi por USB serie a 115 200 baudios.
	**DSSERVO RDS3218** (yaw)	270°. Sin pulso mantiene la posición. No tiene tope mecánico: los límites de software protegen el cableado.
	**2 × TowerPro MG996R** (pitch y roll)	~10 kgf·cm a 6 V. **Sin pulso quedan flojos**: la cabeza cae si se corta el pulso o los 6 V.
	**Buzzer pasivo MH-FMD**	Frecuencia variable (melodías). Trae su transistor PNP: VCC a 3V3, activo en bajo.
	**Powerbank Xiaomi PB2020MI**	Alimenta la Pi: 20 000 mAh, 5 V / 3 A por USB. Dura ~9 h.
	Fuente de laboratorio	Servos a 6,0 V con límite de corriente en 4–5 A (uso normal 2–3 A; un servo trabado pide 2,5–4,2 A).



Componentes reales con sus datos y conexiones (la foto de la cámara es de referencia). Se regenera con `scripts/diagrama_componentes.py`.

Diagrama eléctrico

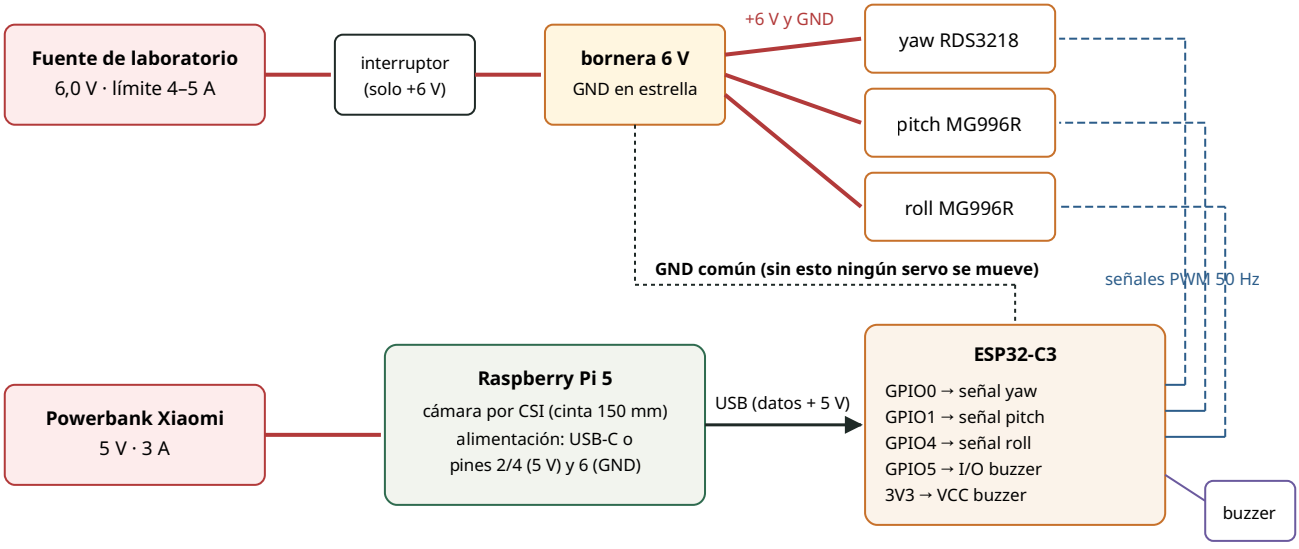


Figura 4. Dos fuentes separadas, unidas solo por el GND. La ESP32 se alimenta por el USB de la Pi; los servos solo toman señal de la ESP32.

Pines de la ESP32-C3

Pin	Uso	Nota
GPIO0	servo yaw	PWM 50 Hz, 14 bits
GPIO1	servo pitch	mismo timer que yaw
GPIO4	servo roll	antes GPIO3 (junto al 2, de strapping)
GPIO5	buzzer (I/O)	timer LEDC aparte; antes GPIO10, que dejó de sonar con la cabeza armada
20 / 21	UART0 (USB serie)	reservados
2 / 8 / 9	strapping	**no usar**: definen el modo de arranque
18 / 19	USB nativo	no usar

Encendido y apagado

Paso	Encender	Apagar
1	Powerbank (Pi)	6 V de los servos
2	Cabeza cerca de HOME	Powerbank
3	6 V de los servos	—

> **¿Por qué ese orden?**
> Los servos no tienen realimentación: el primer pulso los lleva a HOME a toda velocidad. Si la ESP32 ya está mandando el pulso de HOME cuando llegan los 6 V y la cabeza está cerca, el salto es chico. Al apagar, sostener la cabeza: los MG996R quedan flojos.

> **¿Por qué fuente de laboratorio + powerbank? (decisión del 14/10)**
> Los servos generan picos de corriente y ruido; si comparten fuente con la Pi, la Pi puede reiniciarse por caída de tensión. Con dos fuentes separadas la Pi queda aislada. La batería LiPo y los reguladores salieron del diseño: la consigna pide "alimentación propia", no batería.

5 Los servos

Un servo de hobby recibe un pulso cada 20 ms (50 Hz); el **ancho** del pulso indica el ángulo. No informan su posición: el control es en lazo abierto, por eso cada servo se midió en el banco (`pruebas_servos/`).

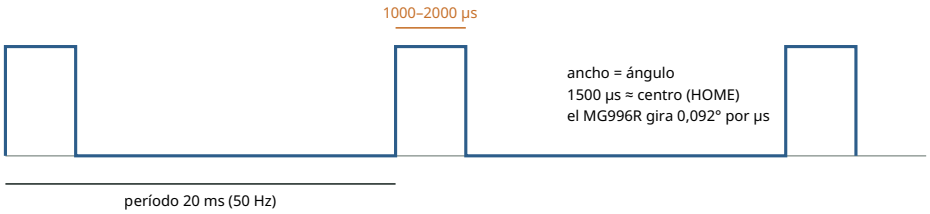


Figura 5. Señal de un servo: la ESP32 la genera con el periférico LEDC, sin usar la CPU.

Conversión de grados a microsegundos

> **Fórmula (`firmware/keepon_fw/nucleo.cpp`, parámetros de `neck.servos`)**
> pulso [µs] = centro + signo × ángulo / (grados por µs), recortado a [min, max].
> Ejemplo, pitch a +30°: 1500 + (−1) × 30 / 0,092 = **1174 µs**, que coincide con lo medido en el robot montado.

Eje	Servo	centro	grados/µs	min / max [µs]	signo	Observaciones
yaw	RDS3218	1500	0,130 / 0,136	950 / 2020	−1	asimétrico: un valor por lado de 1500 µs; ±70° ≈ 960 / 2010 µs
pitch	MG996R	1500	0,092	1100 / 1800	−1	pulso mayor = la cara baja; +30° = 1174 µs, −20° = 1717 µs
roll	MG996R	1500	0,092	1000 / 2000	+1	+217 µs = +20°; signo **verificado montado** el 08/11 (estaba al revés)

El **signo** deja la misma convención para los tres ejes (yaw + = derecha, pitch + = arriba, roll + = horario visto desde atrás) aunque cada servo esté montado en un sentido distinto. Los límites en µs son más amplios que los recorridos y protegen la mecánica si alguna vez llega un ángulo fuera de rango.

Historia de cada eje

Eje	Qué se probó	Resultado
yaw	Dos RDS3218	El n.º 1 vibraba quieto cerca de 1500 µs (≈ 0,5 A); al intercambiarlos la vibración siguió al servo → descartado. Queda el n.º 2.
pitch	HS-7955TG (09/10–05/11) → MG996R nuevo	El HS es fuerte (18 kgf·cm) y sin pulso se mantiene firme, pero metía mucho ruido en la alimentación. El grupo pasó a un MG996R nuevo.
roll	MG996R n.º 1 (13/10–05/11) → MG996R nuevo	El n.º 1 tenía una banda muerta de ~10°: de +25° a 0° quedaba en +10°. Se compensó en el firmware sumando 151 µs según el sentido del movimiento (`compensacion_us`). El nuevo vuelve a 0 sin compensación. El 08/11 se cambió por otro MG996R igual (cero en 1500 µs); vibra solo girando a la derecha en la zona negativa (−20…0), con cualquier perfil: probable servo o carga.

Vibración al moverse lento

Los tres servos vibran cuando avanzan despacio (de a saltitos, por la banda muerta) y no vibran quietos. Bajar la velocidad y la aceleración lo empeoró, y en el roll **los saltos de aceleración también hacían vibrar**. Por eso el robot hace **movimientos rápidos, pocos y sin saltos de aceleración** (curva S, capítulo 6; mirada por objetivo, capítulo 12).

6 Control de los motores

La Pi manda **referencias** de ángulo; la ESP32 genera la trayectoria suave hasta ellas, 100 veces por segundo, respetando límites de posición, velocidad y aceleración. Así ningún comportamiento puede dar un golpe al cuello.

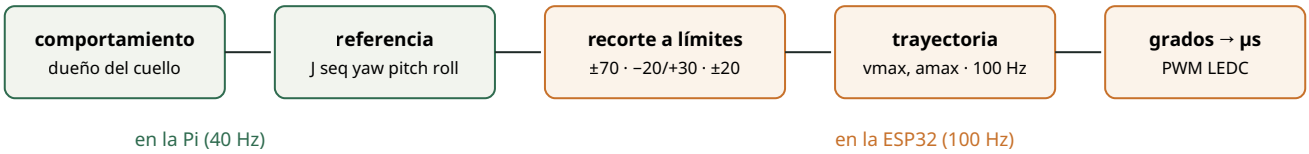


Figura 6. Cadena de control de un eje. Una referencia nueva **reemplaza** a la anterior: no se encolan movimientos.

Perfil de velocidad



Cada eje sigue un perfil **trapezoidal**: acelera con `amax` hasta `vmax`, avanza y frena a tiempo para llegar exactamente al objetivo. Si el recorrido es corto, el trapecio se vuelve triángulo (no llega a `vmax`).

El cálculo es discreto y exacto (`keepon/trajectory.py`, función `_v_max_to_stop`): garantiza que las posiciones enviadas nunca superen `amax` y que el último paso caiga justo en el objetivo. El firmware tiene una copia en C++.

Curva S (desde el 08/11): la posición que va al servo es el **promedio del trapecio en una ventana** de `suavizado_s`. Así la aceleración cambia en rampa en vez de saltar: el jerk del yaw bajó de 60 000–104 000 a ≤ 12 000 °/s³ (≤ 2·amax/ventana), sin pasar `vmax`, `amax` ni el objetivo. Cada movimiento dura ~0,1 s más.

Eje	vmax [°/s]	amax [°/s²]	curva S [s]
yaw	180	600	0,10
pitch	120	400	0,10
roll	250	900	0,15

Ejemplo: girar 25° en yaw lleva ≈ 0,5 s (0,49 s en el simulador). Los valores del roll están en prueba.

Seguridad: watchdog y STOP

Mecanismo	Qué hace	Por qué
Watchdog 500 ms	Si la ESP32 no recibe un comando válido en 500 ms, corta el sonido, frena con `amax` y mantiene la postura. La Pi manda `PING` cada 200 ms si no mandó otra cosa.	Si la Pi se cuelga o se desconecta el USB, el robot no queda moviéndose.
STOP enclavado	Frena suave, corta el sonido y rechaza movimientos y sonidos hasta `RESUME`.	Orden del operador con la máxima prioridad; enclavado para que ningún comportamiento lo libere solo.
Pulso permanente	En STOP y watchdog se sigue mandando el último pulso a los tres servos.	El MG996R sin pulso queda flojo y la cabeza caería.
Arranque en HOME	Al encender, el firmware arranca en HOME y avisa con `HELLO 0`.	Sin realimentación, la única postura conocida es la que se manda primero.

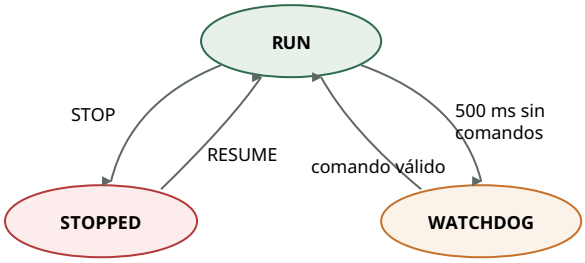
7 Firmware y protocolo serie

El firmware (`firmware/keepon_fw/`) es una copia en C++ del ESP32 simulado en Python. 36+ tests compilan el núcleo del firmware en la notebook y comparan sus respuestas y posiciones con el simulador paso a paso.

Archivos

Archivo	Contenido
`keepon_fw.ino`	Capa de hardware: serie, LEDC de servos y buzzer
`nucleo.cpp`	Lógica portable: protocolo, trayectoria, estados, sonidos, grados → µs
`generado.h`	Generado desde `config.yaml` y las tablas de sonido (no se edita a mano)

Estados del firmware



Comandos (Pi → ESP32)

Comando	Efecto
`HELLO 1`	Abre la sesión
`J seq y p r`	Nueva referencia articular (reemplaza)
`S seq NOMBRE`	Reproduce un sonido; avisa `DONE` al terminar
`CANCEL seq SOUND`	Corta el sonido
`PING seq`	Keepalive; responde `STATE`
`STOP` / `RESUME`	STOP enclavado / liberarlo
`HOME seq`	Va a HOME con rampa
`VOL seq PCT`	Volumen 0–100 % (firmware v2)

Líneas ASCII de hasta 64 caracteres a 115 200 baudios. Cada comando lleva un número de secuencia (`seq`) para asociar la respuesta. Las respuestas son `ACK` , `DONE` , `ERR` , `STATE` y `HELLO` .

Ejemplo de una sesión

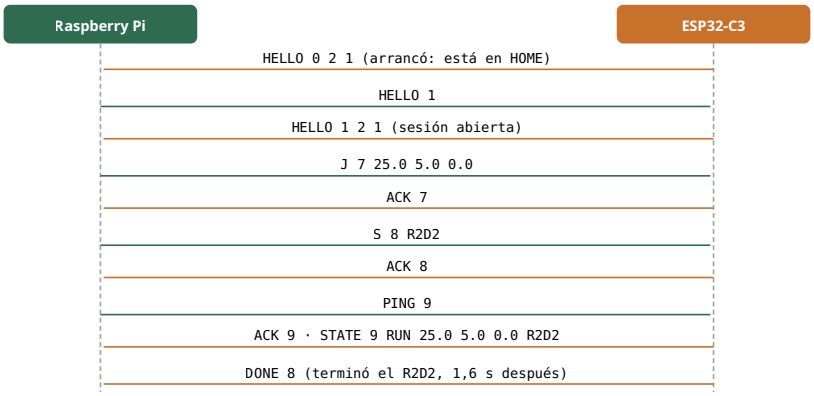


Figura 7. Si en plena sesión llega un `HELLO 0` , la ESP32 se reinició: la Pi cancela todo, toma HOME como referencia, reabre la sesión y reenvía el STOP si estaba activo.

> **Resultados en la ESP32 real (14/10)**
>
> 30 de 30 pruebas OK. Latencia PING → ACK de ida y vuelta: **4,4 ms** (máx. 4,9). Un movimiento a 70/30/−20 llega en 0,68 s (el simulador dice 0,69 s). Watchdog a los ~490 ms. Ocupa 304 KB de flash (23 %) y 14 KB de RAM (4 %).

8 Arquitectura del software

Todo corre en la Pi, en Python puro. ROS 2 se usa solo para leer la cámara (`camera_ros`). La percepción va en 2 procesos aparte y el lazo de decisión y control corre a 40 Hz.

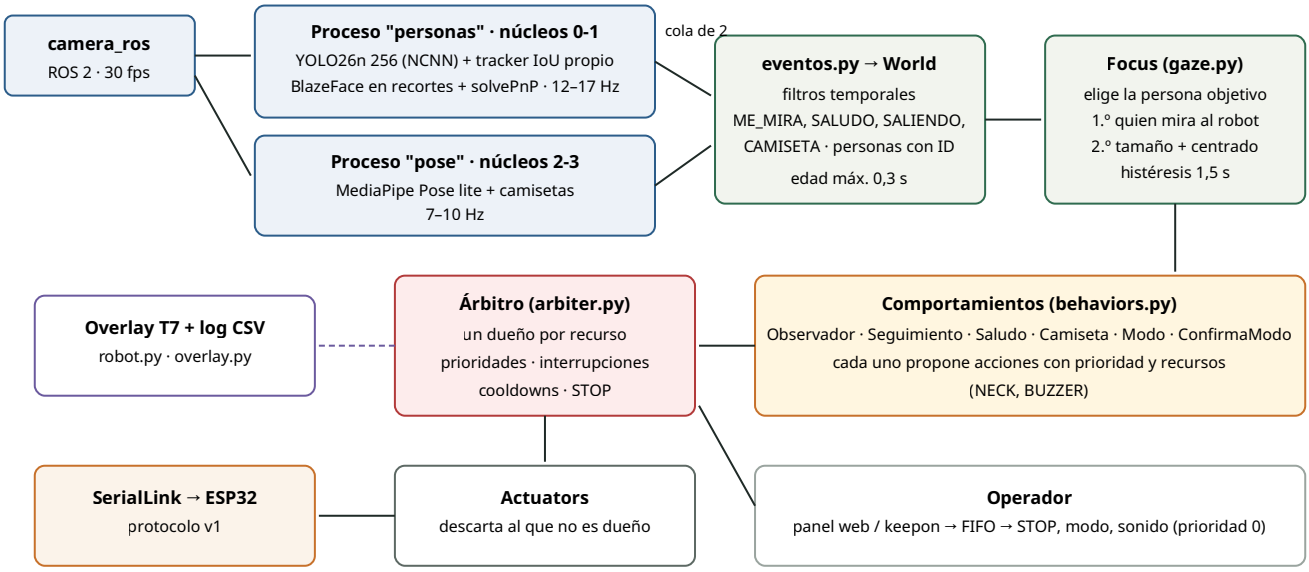


Figura 8. Flujo de datos. Lazo principal en `keepon/robot.py` a 40 Hz; la percepción manda siempre el último resultado y nunca bloquea el lazo.

Módulo	Qué hace
`perception.py`	Los 2 procesos de percepción, fijados a núcleos. Mandan su resultado por una cola de tamaño 2 que descarta lo viejo.
`tracker.py`	Tracker por solapamiento de cajas (IoU), propio: mantiene el ID de cada persona entre cuadros.
`eventos.py`, `camiseta.py`	Pasan de percepción cruda (cuadro a cuadro) a eventos filtrados en el tiempo.
`world.py`	Modelo del mundo: personas con ID, caja, rostro, eventos activos y antigüedad del dato.
`gaze.py`	Control de la mirada y foco de atención.
`behaviors.py`, `arbiter.py`, `actuators.py`	Comportamientos, árbitro y filtro de comandos por dueño.
`trajectory.py`, `protocol.py`, `link.py`, `sim_esp32.py`	Trayectorias, protocolo, enlace serie real o simulado, y la ESP32 simulada (especificación del firmware).
`robot.py`, `overlay.py`	Lazo principal, resincronización si la ESP32 se reinicia, log CSV y overlay de monitoreo.

9 Percepción

Modelos preentrenados (permitidos por la consigna, con su fuente) y todo lo demás propio. El reto fue que entre todo en la CPU de la Pi a ≥ 10 Hz.

Qué	Cómo	Fuente / por qué
Personas	YOLO26n exportado a NCNN, entrada 256 px, confianza 0,4	Ultralytics. A 480 px tardaba 79 ms y saturaba la CPU; a 256 px tarda 18 ms y detecta a 2 de 2 personas a 1,5 m en el 100 % de los cuadros.
ID entre cuadros	Tracker IoU propio	El ByteTrack de Ultralytics costaba ~68 ms por cuadro en la Pi.
Rostro	MediaPipe BlazeFace (6 puntos) sobre un recorte de 192 px de la cabeza	Google MediaPipe. Face Landmarker (478 puntos) tardaba ~49 ms por cara y sobre la imagen completa encontraba solo 0,4–0,7 de 2 rostros.
¿Me mira?	solvePnP con los 6 puntos → orientación de la cara	Ángulo entre el eje de la cara y la línea cara → cámara < 15° (`looking_deg`).
Pose (brazos, hombros)	MediaPipe Pose lite, hasta 2 personas	Google MediaPipe. Para el SALUDO y para ubicar el torso (camisetas).

Cómo se llegó a ≥ 10 Hz

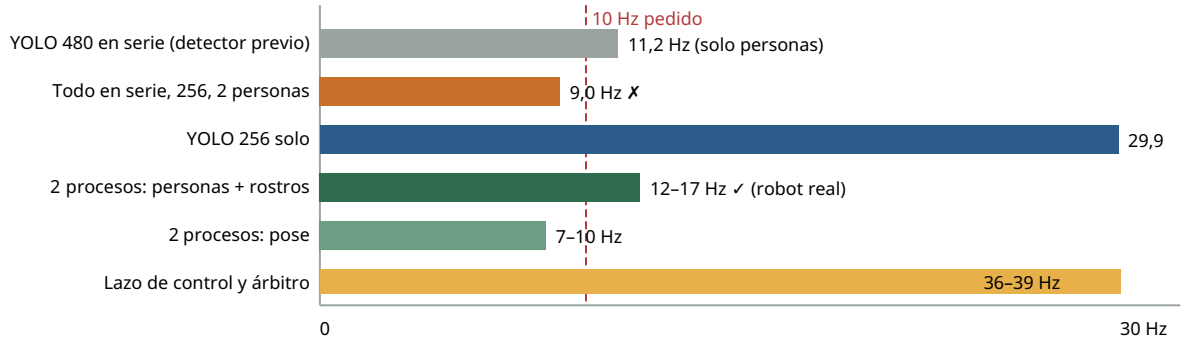


Figura 9. Frecuencias medidas en la Pi 5 (`docs/ESTADO_PI.md`). La barra del lazo está fuera de escala (36–39 Hz).

¿Por qué procesos y no hilos?

NCNN y MediaPipe no liberan el GIL de Python: en hilos tardaban casi lo mismo que en serie (2,04 s contra 2,11 s). En procesos separados, cada uno en 2 núcleos fijos, corren de verdad en paralelo. NCNN y MediaPipe reubican sus hilos, así que el código los vuelve a fijar cada 50 cuadros.

Cámara sin calibrar

Los intrínsecos (fx = 948 → campo de 37,3°) son de otra cámara y el centro óptico se tomó como el centro de la imagen (el de la calibración ajena corría la mirada ~6,5°). Falta calibrar con un tablero: afecta sobre todo a ME_MIRA.

10 Eventos: definición operativa

Un evento no se dispara por un cuadro suelto: cada uno tiene un umbral y un filtro temporal con histéresis (`keepon/eventos.py`, parámetros en `eventos:` y `camiseta:` de `config.yaml`).

Evento	Condición en cada cuadro	Filtro temporal
PERSONA	Caja de YOLO de clase persona con confianza ≥ 0,4, asociada a un ID por el tracker.	Si el objetivo desaparece menos de 1 s, se mantiene; después se libera (`lost_hold_s`).
ME_MIRA	Desvío entre el eje de la cara y la línea cara → cámara < 15°.	Se activa tras 0,5 s seguidos mirando y se suelta tras 0,3 s sin mirar.
SALUDO	Muñeca al menos 0,1 hombros por encima del hombro, que oscila de costado con amplitud ≥ 0,25 anchos de hombros (medido en hombros para no depender de la distancia).	2 ciclos (cambios de sentido) dentro de 2,5 s → un evento. Para contar otro, la mano tiene que bajar 0,4 s: un gesto = una respuesta. Cooldown de 5 s por persona.
CAMISETA_*	Clase del torso según reglas de color y patrón (capítulo 11).	6 cuadros seguidos para confirmar y 6 para soltar, por persona. La respuesta suena **una vez por aparición** de cada persona (por ID del tracker; si sale más de ~1 s, vuelve con otro ID).
SALIENDO	A menos del 15 % del ancho del borde y alejándose a ≥ 80 px/s, o el yaw a menos de 4° de su límite con la persona más allá del centro.	Se activa si dura 0,2 s y se suelta tras 0,6 s sin la condición.

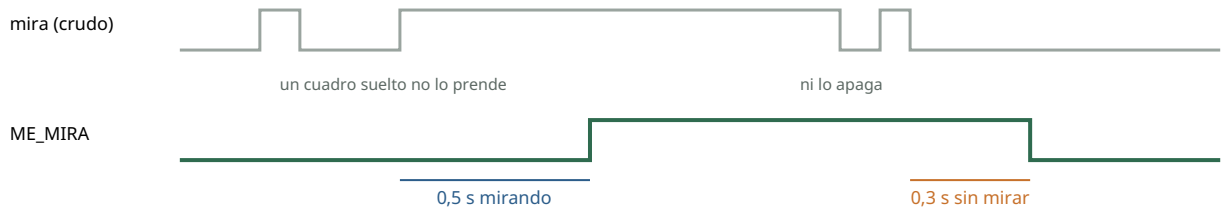
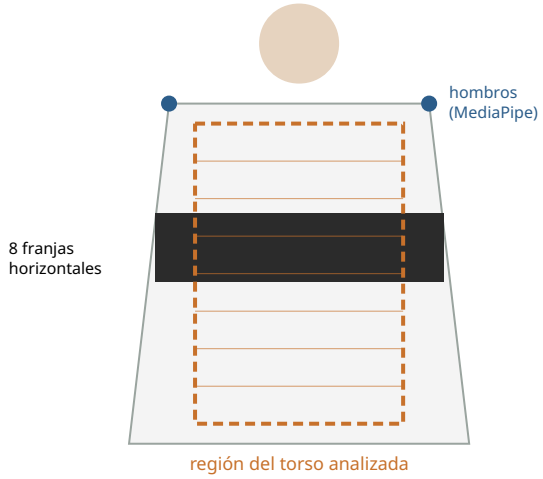


Figura 10. Histéresis temporal de ME_MIRA.

> **ME_MIRA casi no se activaba en las pruebas**
> Con la mirada corrida 8° hacia abajo, el ángulo medido dio una mediana de 8° (30 de 42 lecturas por debajo de 15°). Falta repetirlo; con la cámara calibrada debería mejorar. De esto dependen el modo Tímido y el Atrevido.

11 Detector de camisetas

Reglas de color y patrón sobre el torso, sin red entrenada (la guía lo permite). Desarrollo propio: `keepon/camiseta.py`, corre en el proceso de pose.



1. Ubicar el torso. Con los hombros de MediaPipe: desde 0,15 hasta 1,1 anchos de hombros debajo de ellos, descartando un 10 % a cada lado (brazos). Se exigen hombros visibles, al menos 40 px de ancho y 40 % del torso dentro del cuadro.

2. Clasificar cada píxel en HSV (rojo, azul, blanco, oscuro) con umbrales medidos con la NoIR sobre las prendas reales.

3. Buscar el patrón.

Clase	Regla
Olimpia	8 franjas horizontales: una oscura (brillo ≤ 110) con franjas blancas (saturación ≤ 60) arriba y abajo, al menos 1,6 veces más claras.
Cerro	8 columnas: al menos 2 rojas y 2 azules (≥ 1/4 del ancho cada color) separadas por un borde vertical, y ≥ 20 % de píxeles de cada color.
Nada	Lo demás: lisas rojas, azules o blancas no cumplen ningún patrón.

> **¿Por qué no exigir rayas para Cerro?**
> La titular 2026 de Cerro es mitad roja y mitad azul, sin rayas: la regla pide rojo **y** azul en partes grandes separados por un borde vertical, así detecta las dos. Además, con la NoIR el azul marino de Cerro se ve violeta (H 116–145) y la franja negra de Olimpia, marrón violácea.

Resultados con la cámara del robot (04/11)

Toma (60 s, a 1,2 / 1,8 / 2,5 m)	Detectada tras el filtro de 6 cuadros	Falsos positivos
Olimpia titular	**82 % (99 % desde los 10 s)**	0 %
Cerro 2026, mitad y mitad	**98 %**	0 %
Remera roja lisa	—	**0 %**
Remera azul marino lisa	—	**0 %**
Remera blanca lisa	**Solo probada con una imagen sintética (test). Falta una real.**	

Limitación conocida: la camiseta del Barcelona es casi igual a la de Cerro por color y patrón.

12 Control de la mirada y foco de atención

Del píxel al ángulo

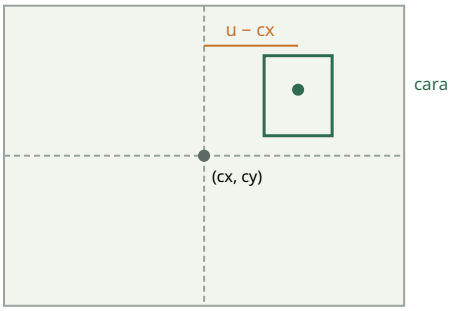


imagen 640 × 480 · campo ≈ 37° × 28°

El error angular de la cara respecto del centro óptico es:

> error_yaw = atan((u − cx) / fx) · error_pitch = atan((v − cy) / fy)

Con fx ≈ 948 px, 1° equivale a ~16,5 px en el centro de la imagen. La cámara apunta 8° por debajo de la cara (`mirar_abajo_deg`) para que entre la camiseta en el cuadro.

Mirada por objetivo (modo actual, desde el 05/11)

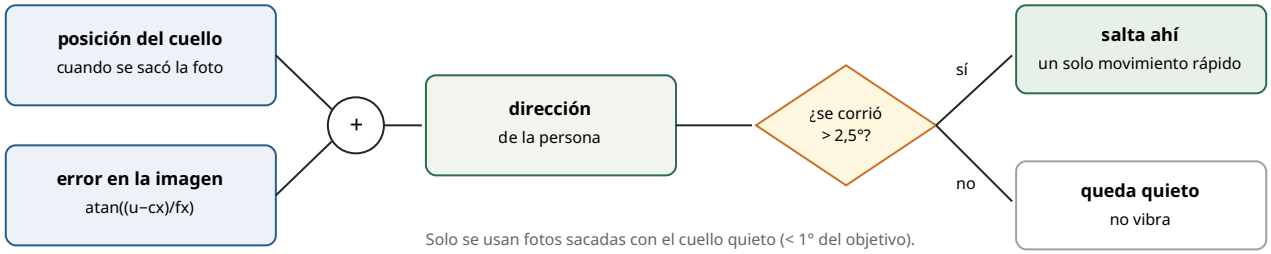


Figura 11. Usar la posición del cuello **en el momento de la foto** compensa el atraso de la percepción (~100–135 ms): en simulación, un salto a 25° con 150 ms de atraso ya no se pasa a 29° y llega derecho a 25°.

> **¿Por qué se cambió el control? (05/11)**
> Antes (modo `velocidad`) la referencia avanzaba un poquito en cada ciclo según el error: velocidad = kp × (|error| − zona muerta), con saturación y suavizado EMA. Eso hacía que los servos se arrastraran despacio y vibraran. Con la mirada por objetivo, en el simulador los cambios de referencia bajaron de 25 a 0 con la persona quieta y de 300 a 29 con la persona caminando. El modo `velocidad` queda como respaldo en `config.yaml`.

Foco de atención: a quién mirar

Regla	Detalle
1. Quien mira al robot	Prioridad estricta: una persona con ME_MIRA gana siempre a una que no mira.
2. Si nadie mira	Puntaje = 0,6 × tamaño + 0,4 × centrado (la más cercana y central).
3. Histéresis	Un candidato tiene que ser mejor durante 1,5 s seguidos para quitarle el foco al actual: evita saltos constantes.
4. Pérdida breve	Si el objetivo desaparece menos de 1 s se mantiene la mirada; después se libera y vuelve el Observador.

13 Comportamientos y árbitro

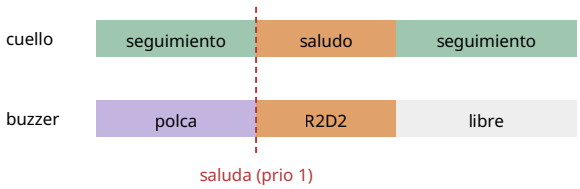
Hay dos recursos exclusivos: el **cuello** (NECK) y el **buzzer** (BUZZER). Cada comportamiento propone acciones; el árbitro decide quién es dueño de cada recurso en cada instante. Es desarrollo propio (`keepon/arbiter.py`).

Prio.	Comportamiento	Recursos	Lo puede interrumpir	Qué hace
0	**OPERADOR**	todos	nadie	STOP y HOME enclavados hasta RESUME, sonidos, beep de cambio de modo (el último sonido pedido gana).
0	**EJES**	cuello	nadie	Demostración de los 3 ejes (~15 s, botón del panel); al terminar vuelve a HOME y retoma el Observador o el seguimiento.
1	**SALUDO**	cuello + buzzer	solo el operador	3 cabeceos de 12° + R2D2. Cooldown 5 s por persona.
2	**CAMISETA**	buzzer	operador o saludo	Polca (Olimpia) o abucheo (Cerro), una vez por aparición de cada persona.
3	**MODO**	cuello o buzzer	los más importantes	Reacción del modo de personalidad (capítulo 14).
4	**SEGUIMIENTO**	cuello	cualquiera más importante	Mantener la mirada en la persona objetivo (continuo).
5	**OBSERVADOR**	cuello	cualquiera más importante	Explorar sin personas (continuo): centra, barre ±45° a 25 °/s de media con mínimo jerk y cada 2 barridos hace una sacada a un punto de interés con roll ±5°.

Reglas del árbitro

1. **Exclusividad:** cada recurso tiene a lo sumo un dueño.
2. **Prioridad:** número menor = más importante. Una propuesta solo le quita un recurso a un dueño menos importante *y* que acepte ser interrumpido por ella.
3. **Atomicidad:** se conceden todos los recursos pedidos o ninguno. Si a una acción le quitan un recurso, se cancela entera.
4. **Vencimiento y cooldown:** las propuestas viejas o en cooldown se descartan; el cooldown cuenta desde que la acción *empieza*.
5. **Continuas y episódicas:** seguir y observar retienen el cuello mientras se vuelvan a proponer; saludo y polca, hasta que terminan.
6. **STOP** cancela todo, incluso lo no interrumpible, y queda enclavado hasta RESUME.

Ejemplo: Olimpia + saludo



• La polca y el seguimiento **conviven**: usan recursos distintos. • El saludo pide cuello + buzzer: le quita el buzzer a la polca (prio 2, interrumpible por 1) y el cuello al seguimiento. La polca se cancela. • Al terminar el saludo, el seguimiento retoma el cuello.

> **Una sola orden coherente al cuello**
>
> `Actuators` descarta cualquier comando de quien no es dueño del recurso: nunca hay dos controladores escribiendo al cuello. La curiosidad y los gestos de los modos se calculan dentro del dueño del cuello, sumados a la mirada. En el escenario simulado de 30 s se comprueba que hay cero violaciones.

Curiosidad (extra)

Si la persona seguida queda quieta 3 s (su dirección se mueve menos de 5°), el robot inclina la cabeza 15° en roll durante 2 s, alternando el lado, y no lo repite antes de 8 s.

14 Modos de personalidad

Se cambian en cualquier momento desde el panel o con `keepon n|t|a|m`; cada cambio se confirma con un beep distinto. El modo inicial es Neutro.

Modo	Disparador	Reacción	Beep
Neutro	—	Solo sostiene el contacto visual (seguimiento).	0,25 s
Tímido	ME_MIRA	Aparta la mirada 25° al costado con más lugar, 12° abajo y 10° de roll; la sostiene 2,5 s y vuelve a espiar en 3 s con una curva suave.	0,35 s
Atrevido	ME_MIRA	Levanta el mentón 10°, 3 veces (período 1,2 s), sin dejar de seguir en yaw.	0,35 s
Maleducado	SALIENDO	Silbido de piropo, una vez por salida y como mucho uno cada 40 s entre todos.	0,45 s

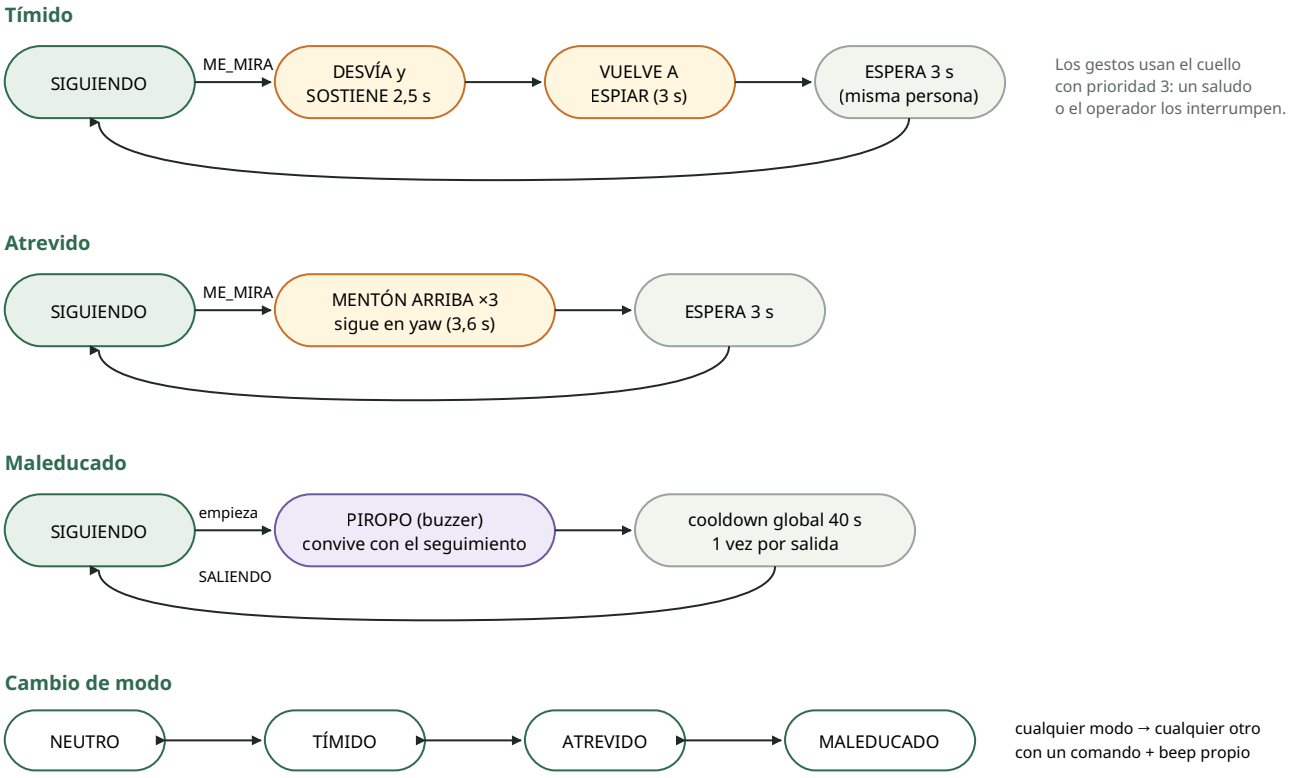


Figura 12. Diagramas de estados de los modos. Las esperas son cooldowns por persona (Tímido, Atrevido) o global (piropo).

Estado de las pruebas en el robot

El cambio de modo desde el panel, con su beep, y los modos Tímido, Atrevido y Maleducado se probaron en el robot: están en el video (1:33–2:05). En las primeras pruebas (03–05/11) Tímido no se activaba porque ME_MIRA casi no se prendía; el umbral de 15° sigue sin validarse con la cámara calibrada. El beep corta cualquier sonido que esté sonando (polca, saludo u otro del operador): el último pedido gana.

15 Buzzer y sonidos

Un buzzer **pasivo** suena a la frecuencia de la señal que recibe, así que puede tocar melodías. Cada sonido es una tabla de segmentos `{frecuencia inicial, frecuencia final, ms}`: nota fija, barrido lineal o silencio.

Sonido	Duración	Uso
R2D2	1,6 s	respuesta al saludo
POLCA	7,1 s	camiseta de Olimpia
ABUCHEO	2,5 s	camiseta de Cerro
PIROPO	1,2 s	modo Maleducado
DESAPROBACION	0,8 s	disponible (opcional)
POLCA_CERRO	6,0 s	extra
BEEP_*	0,25–0,45 s	confirmación de cada modo



Los sonidos se diseñaron y escucharon en `pruebas_buzzer/simulador_sonidos.html`, que genera las tablas en C para el firmware.

La polca de Olimpia

Transcrita a mano de la partitura (`pruebas_buzzer/notas_polca_de_olimpia/`): compases 1 a 8 en 6/8, negra con puntillo = 135, Fa mayor (corchea de 148 ms, 7,104 s en total). Va una octava más arriba que lo escrito (659–1397 Hz), porque más abajo el buzzer suena débil. Cada nota deja 15 ms de silencio para separar las repetidas.

> **Decisiones del buzzer**
> • Timer LEDC separado del de los servos: cambiar de nota no altera el PWM de los servos.
> • Módulo MH-FMD a 3V3 (no 5 V): su transistor PNP solo se apaga si el I/O llega a la tensión de VCC.
> • El volumen se regula con el ciclo de trabajo del PWM (50 % = máximo), en tiempo real con `VOL`.
> • Fragmento de 8 compases y no la polca completa: la completa dura más de un minuto y bloquearía el buzzer.
> • Un sonido intermitente resultó ser un cable jumper con falso contacto en el cuello: hay que soldarlo y precintarlo.

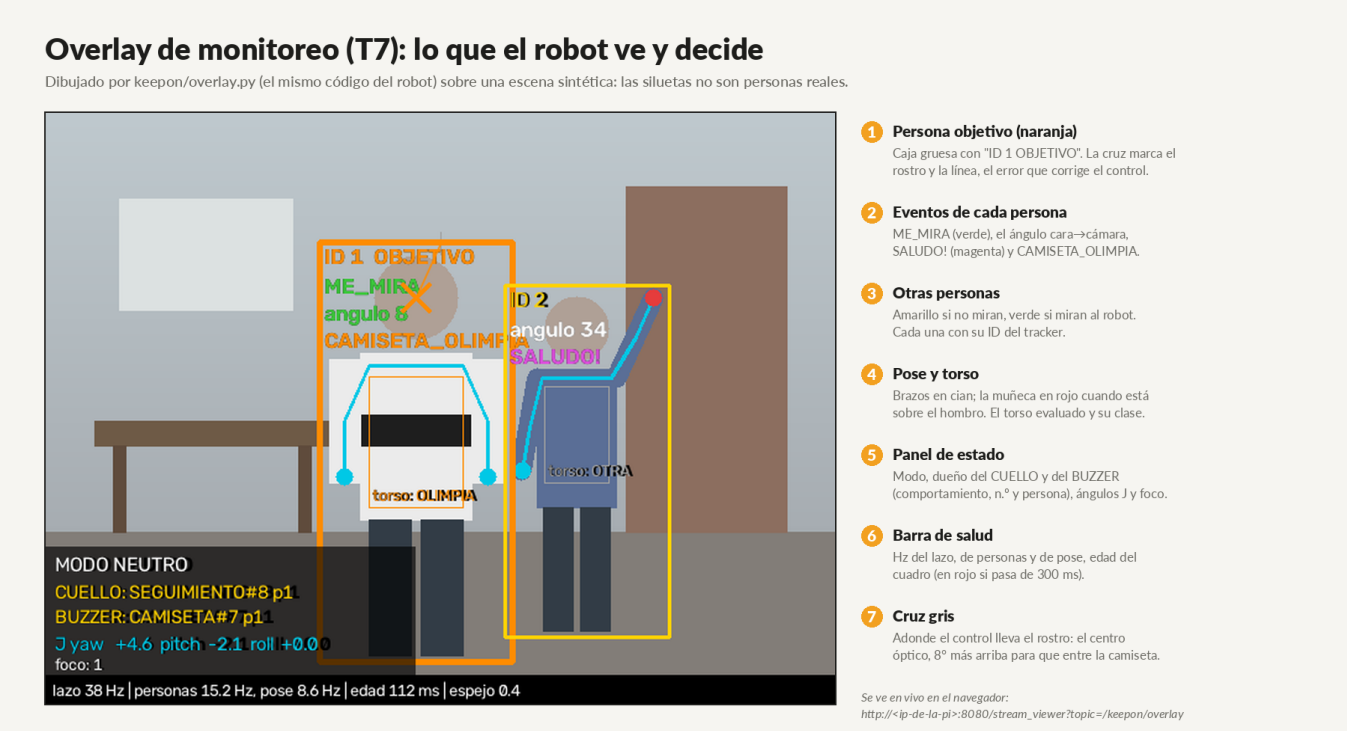
16 Monitoreo, log y operación

Overlay de monitoreo (T7)

El video de la cámara con información dibujada encima, publicado en `http://<ip-de-la-pi>:8080/stream_viewer?topic=/keepon/overlay`:

Muestra	Detalle
Personas con su ID	Caja, ID del tracker y eventos activos de cada una (mira, saludo, camiseta, saliendo).
Persona objetivo	Resaltada, con el ángulo de su mirada (`angulo N`) y el torso evaluado para las camisetas.
Modo y dueños	Modo actual y el comportamiento dueño del CUELLO y del BUZZER.
Ángulos comandados	Referencias de yaw, pitch y roll enviadas a la ESP32.

Además, cada transición de estado (quién toma o suelta un recurso, eventos, comandos) se guarda con marca de tiempo en `logs/run_*.csv`, y cada pocos segundos se escribe una línea ESTADO con las frecuencias y la edad del cuadro.



El overlay dibujado por `keepon/overlay.py` sobre una escena sintética (las siluetas no son personas reales).

Cómo se opera

Qué	Cómo
Encender / apagar el robot	Panel del celular `:8081` (botones Encender / Apagar), o `keepon arrancar` / `keepon parar`. Al parar vuelve a HOME.
Modos	`keepon n` · `t` · `a` · `m` (Neutro, Tímido, Atrevido, Maleducado)
STOP y HOME	`keepon stop` · `keepon home` (los dos quedan enclavados) · `keepon resume` los libera
Demostración de los ejes	`keepon ejes` o el botón **Ejes** del panel: yaw, pitch y roll en ~15 s
Sonidos y volumen	`keepon sonido POLCA` · `keepon volumen 0-100`
Ver qué pasa	`keepon estado` · `keepon log`
Demo	`keepon demo-on`: arranca solo al encender la Pi (por defecto está apagado porque la Pi se usa en otros proyectos).

> **¿Por qué un panel web y no una app?**
> Una página instalable (PWA, `scripts/panel_web.py`) anda en cualquier celular sin instalar nada. Corre como servicio aparte siempre prendido, así que sirve también con el robot apagado. Solo acepta una lista cerrada de comandos: desde la página no se puede ejecutar otra cosa en la Pi.



El panel del celular (captura de la app real con un estado de ejemplo) y qué hace cada grupo de botones.

17 Por qué se decidió cada cosa

Resumen de `docs/DECISIONES.md`. Estas decisiones no se rediscuten sin datos nuevos.

Decisión	Alternativa descartada	Por qué
Todo en la Pi, en Python puro	ROS 2 completo, o procesar en la notebook	Más simple de explicar y depurar; ROS 2 solo para leer la cámara, que ya funcionaba así en la Pi.
Mantener la ESP32 para los servos	Pulsos desde la Pi	La consigna pide microcontrolador + protocolo; el watchdog es independiente de Linux y el PWM no depende de la carga de la CPU.
Pi dentro de la cabeza	Pi en la base	La cinta de la cámara mide ~150 mm y no llega a la base.
Fuente de laboratorio + powerbank	LiPo con reguladores	Dos fuentes separadas aíslan la Pi del ruido y los picos de los servos.
YOLO26n a 256 px en NCNN	480 o 320 px	4,3 veces más rápido que a 480 y detecta igual a 1,5 m.
2 procesos fijados a núcleos	Hilos	NCNN y MediaPipe no liberan el GIL: con hilos no hay paralelismo.
BlazeFace sobre recortes	Face Landmarker (478 puntos)	~49 ms por cara y no encontraba los rostros lejanos en la imagen completa.
Tracker IoU propio	ByteTrack de Ultralytics	ByteTrack costaba ~68 ms por cuadro en la Pi.
Camisetas con reglas de color y patrón	Red entrenada	Sin datos para entrenar; las reglas se explican y ajustan con las prendas reales. La guía las permite.
Movimientos rápidos y pocos	Movimientos lentos y continuos	Los servos vibran moviéndose despacio; bajar vmax/amax/kp empeoró.
Curva S en todos los movimientos (08/11)	Trapecio solo	La aceleración cambia en rampa (jerk de 60 000+ a 12 000 °/s^3); en el roll, sin curva S vibraba más.
Observador «exploracion» (08/11)	De punto en punto con pausas	Es lo que pide la Tabla 1 (barridos lentos, sacadas, roll para encuadrar); el anterior se veía «para y gira».
Polca una vez por aparición (08/11)	Cooldown de 25 s	Con la camiseta a la vista la polca se repetía (4 veces en 90 s).
HOME enclavado y Ejes con prioridad 0 (08/11)	HOME por fuera del árbitro	Los comportamientos pisaban el HOME en el ciclo siguiente; ahora queda hasta RESUME.
Mirada por objetivo	Control P en velocidad	Compensa el atraso de la cámara y deja el cuello quieto entre movimientos.
2 × MG996R nuevos en pitch y roll	HS-7955TG y MG996R con zona muerta	Decisión del grupo (05/11): el HS metía ruido y el MG996R viejo tenía ~10° de zona muerta.
Firmware = copia del simulador	Escribir el firmware aparte	Los tests comparan los dos paso a paso: el comportamiento se prueba en la notebook.
Fragmento de 8 compases de la polca	Polca completa	La guía pide "un fragmento reconocible"; la completa ocuparía el buzzer más de un minuto.
Servicio de demo no activado al encender	Arranque automático siempre	La Pi se usa en otros proyectos y el robot toma la cámara y la ESP32.
Repo público	Repo privado	Para que la cátedra lo lea desde afuera (07/11). No se sube nada personal.

18 Resultados, pruebas y lo que falta

Requisitos de la consigna contra lo medido

Requisito	Medido	Estado
Percepción ≥ 10 Hz	Personas 12–17 Hz · pose 7–10 Hz	**Cumple** en personas; la pose queda justo en el límite.
Latencia evento → respuesta ≤ 300 ms	Edad del cuadro 96–135 ms · lazo 25 ms · serie 4,4 ms	**Estimación ≈ 130–170 ms; falta medirla de punta a punta** (p. ej. SALUDO → R2D2 con el log).
Todo no bloqueante	Firmware sin `delay()`; percepción en procesos; colas que descartan lo viejo	**Cumple**
Recursos exclusivos	0 violaciones en el escenario de 30 s	**Cumple**
Prioridades, cooldowns, histéresis	Tabla de prioridades; saludo 5 s por persona, camiseta una vez por aparición, piropo 40 s; histéresis de foco 1,5 s	**Cumple**
Overlay y log con marca de tiempo	Overlay T7 y log CSV	**Cumple**

Pruebas automáticas

258
tests con pytest

36+
firmware contra simulador, con fuzzing

30/30
pruebas en la ESP32 real

8
fallos de una revisión externa, cada uno con su test de regresión

Además, `scripts/sim_scenario.py` corre un escenario completo de 30 s con personas sintéticas (seguimiento, saludo, camiseta, conflictos, STOP) contra la ESP32 simulada o contra la real.

> **Un bug real que encontraron los tests**
> El firmware daba watchdog y fines de sonido falsos al restar tiempos sin signo (`millis()` contra el instante programado). Se corrigió con una diferencia con signo: 60 s a 40 Hz y 6 minutos de robot real sin ningún WATCHDOG. Cada bug encontrado queda como test de regresión.

Funciones: qué está probado en el robot

Probado en el robot real	Hecho, falta probar en el robot	Falta
Seguimiento con foco e histéresis · Observador «exploracion» · Saludo (3 cabeceos + R2D2) · Olimpia → polca mientras sigue · Cerro → abucheo · lisas sin falsos positivos · cambio de modo con beep · Tímido, Atrevido y Maleducado · dos personas · pérdida y recuperación · STOP · Ejes · polca una vez por aparición · curiosidad · HOME del operador · apagar a HOME · signo del roll · overlay y log · panel del celular (las 8 pruebas del video)	Validar el umbral de ME_MIRA (15°) con la cámara calibrada y más personas	Medir la latencia de punta a punta · calibrar la cámara · remera blanca real · banco del servo de roll

Lo que queda por hacer, por prioridad

#	Tarea
1	Antes de la demo: sincronizar la Pi (`/pi-sync-keepon`), reiniciar el panel y probar la secuencia completa con la luz y las camisetas de la demo.
2	Repetir antes de la demo la secuencia de las 8 pruebas del video, con 2–3 personas.
3	Medir la latencia evento → inicio de respuesta (p. ej. SALUDO → R2D2 con el log).
4	Calibrar la cámara con un tablero y validar el umbral de ME_MIRA (15°).
5	Llevar el servo de roll al banco por la vibración girando a la derecha; si los servos tiemblan en giros rápidos, volver a poner el capacitor de 2200 µF + 100 nF.
6	Ensayo de la defensa con los cuatro integrantes (capítulo 19).

Aportes individuales

Integrante	Desarrollo realizado	Archivos / tareas
Mathias Aguilar	Diseño mecánico y programación en general	Diseño de la estructura (`docs/diseno_v6/` , v6 R4) y programación del robot (`keepon/`)
Héctor Velázquez	Conexiones, soldadura y programación	Cableado y alimentación, firmware de la ESP32 (`firmware/`) y programación (`keepon/` , `scripts/`)
Pablo Otazú	Ensamblado de la parte mecánica	Armado de las piezas impresas del v6 R4 con los servos
Pedro Vera	Modelado en SolidWorks y programación	Modelo en SolidWorks del v6 R4 y programación del robot (`keepon/`)

> **Uso de IA**
> Se usó **Claude Code** (Anthropic) en la terminal para desarrollar la programación (código del robot, firmware, scripts, tests y parte de la documentación) y se pidieron revisiones a **Codex** (OpenAI), guardadas en `docs/revisiones/` . La consigna lo permite si se comprende el proceso y el resultado: en la defensa pueden preguntar cualquier parte en detalle.

> **Dónde está cada cosa en el repo**
> `keepon/` el robot en Python · `firmware/` ESP32-C3 · `config.yaml` todos los parámetros · `tests/` pruebas · `scripts/` arranque, panel, carga de firmware y benchmarks · `pruebas_servos/` , `pruebas_buzzer/` , `pruebas_camisetas/` bancos de prueba · `docs/` documentación: empezar por `docs/README.md` (mapa y recorrido de lectura); `docs/consigna/` , `docs/informe/` , `docs/hardware/` , `docs/diseno_v6/` , `docs/revisiones/` .

19 Para la defensa

La consigna dice que todos los integrantes tienen que poder responder sobre **cualquier parte** del sistema y que la nota individual se puede ajustar según la defensa. Estas son preguntas probables, con una respuesta corta sacada de esta guía; el detalle de cada tema está en los capítulos anteriores.

1. ¿Qué hace la Raspberry Pi y qué hace la ESP32?
La Pi hace toda la inteligencia: percepción, eventos, foco, comportamientos y árbitro (lazo de 40 Hz). La ESP32 solo ejecuta: genera las trayectorias a 100 Hz con límites y curva S, el PWM de los servos y los sonidos, con watchdog y STOP. Se mantuvo la ESP32 porque la consigna pide microcontrolador + protocolo, el PWM no depende de la carga de Linux y el watchdog es independiente de la Pi.

2. ¿Cómo llegaron a ≥ 10 Hz de percepción?
YOLO26n exportado a NCNN con entrada de 256 px (18 ms en vez de 79 ms a 480 px), percepción en 2 procesos fijados a núcleos (NCNN y MediaPipe no liberan el GIL, con hilos no había paralelismo), tracker IoU propio (ByteTrack costaba ~68 ms) y BlazeFace sobre recortes de la cabeza. Resultado: personas 12–17 Hz, pose 7–10 Hz.

3. ¿Qué es el árbitro y cómo resuelve un conflicto?
Asigna los dos recursos exclusivos (cuello y buzzer). Reglas: un dueño por recurso, una propuesta solo le quita un recurso a un dueño menos importante que acepte ser interrumpido, se conceden todos los recursos pedidos o ninguno, y STOP cancela todo. Ejemplo: la polca (prioridad 2, buzzer) convive con el seguimiento (4, cuello); si la persona saluda, el saludo (1, cuello + buzzer) interrumpe a los dos y al terminar el seguimiento retoma el cuello.

4. ¿Cómo evitan que el robot repita las respuestas?
Saludo: cooldown de 5 s por persona y, para contar otro gesto, la mano tiene que bajar 0,4 s. Camiseta: una vez por aparición de la persona. Piropo: una vez por salida (flanco de SALIENDO) y como mucho uno cada 40 s. Los cooldowns cuentan desde que la acción empieza.

5. ¿Cómo se define ME_MIRA?
Con los 6 puntos de BlazeFace y solvePnP se estima hacia dónde apunta la cara; si el ángulo entre ese eje y la línea cara → cámara es menor a 15°, la persona mira. Se activa tras 0,5 s seguidos y se suelta tras 0,3 s (histéresis). Limitación: casi no se activaba porque la cámara no está calibrada con tablero.

6. ¿Cómo detectan el saludo?
Con MediaPipe Pose: la muñeca tiene que estar al menos 0,1 hombros por encima del hombro y oscilar de costado con amplitud ≥ 0,25 anchos de hombros (medido en hombros para no depender de la distancia). 2 cambios de sentido dentro de 2,5 s = un saludo.

7. ¿Cómo detectan las camisetas?
Con reglas propias de color y patrón, sin red entrenada: se ubica el torso con los hombros, se clasifica cada píxel en HSV y se busca el patrón (Olimpia: franja oscura entre franjas blancas; Cerro: rojo y azul en partes grandes separados por un borde vertical). Con la cámara del robot: Olimpia 82 % (99 % desde los 10 s), Cerro 98 %, lisas 0 % de falsos positivos. Limitación: Barcelona se parece a Cerro.

8. ¿Cómo eligen a quién mirar?
Primero a quien mira al robot (ME_MIRA). Si nadie mira, puntaje = 0,6 × tamaño + 0,4 × centrado. Un candidato tiene que ser mejor durante 1,5 s seguidos para quitarle el foco al actual. Si el objetivo se pierde menos de 1 s, se mantiene la mirada.

9. ¿Cómo controlan la mirada?
Mirada por objetivo: dirección = posición del cuello en el momento en que se sacó la foto + error de la cara en la imagen. Si se corrió más de 2,5°, se manda un solo movimiento rápido y el cuello queda quieto. Usar la posición del momento de la foto compensa el atraso de la percepción (~100–135 ms). El control proporcional en velocidad anterior hacía avanzar a los servos despacio y vibraban.

10. ¿Qué es la curva S y por qué la agregaron?
La posición que va al servo es el promedio del perfil trapezoidal en una ventana corta (0,10–0,15 s): la aceleración cambia en rampa en vez de saltar. El jerk del yaw bajó de 60 000+ a 12 000 °/s^3 sin pasar vmax ni amax. En el roll, sin curva S vibraba más.

11. ¿Qué pasa si se corta la comunicación o se reinicia la ESP32?
Si la ESP32 no recibe nada válido en 500 ms (watchdog), corta el sonido, frena y mantiene la postura sin soltar los servos; la Pi manda PING cada 200 ms. Si la ESP32 se reinicia avisa HELLO 0: la Pi cancela todo, toma HOME como referencia, reabre la sesión y reenvía el STOP si estaba activo.

12. ¿Cómo garantizan que nada bloquea?
La percepción corre en procesos aparte y manda resultados por colas de tamaño 2 que descartan lo viejo; el lazo de la Pi nunca espera a la ESP32 (órdenes asíncronas con ACK); el firmware avanza trayectorias y sonidos por pasos temporizados con millis(), sin delay().

13. ¿Cómo pasan de grados a pulso?
pulso = centro + signo × ángulo / (grados por µs), recortado a [min, max]. Ejemplo: pitch a +30° = 1500 + (−1) × 30 / 0,092 = 1174 µs, que coincide con lo medido montado. Cada servo se caracterizó en un banco porque no informan su posición.

14. ¿Por qué dos fuentes de alimentación?
Los servos generan picos de corriente y ruido; compartiendo fuente la Pi podía reiniciarse. Powerbank para la Pi y fuente de laboratorio a 6 V para los servos, unidas solo por el GND (sin GND común ningún servo se mueve). Encender: powerbank y después 6 V; apagar al revés, sosteniendo la cabeza.

15. ¿Qué es desarrollo propio y qué no?
Propio: árbitro, comportamientos, control de la cabeza, tracker, eventos, detector de camisetas, firmware, panel y diseño mecánico. De terceros (citados): YOLO26n (Ultralytics), MediaPipe (Google), OpenCV y camera_ros. IA para desarrollar: Claude Code, y revisiones con Codex.

16. ¿Cómo lo probaron sin el robot?
Primero se escribió el robot completo contra una ESP32 simulada y personas sintéticas. El firmware es una copia en C++ del simulador y 36+ tests los comparan paso a paso. En total 258 tests y un escenario de 30 s con 0 violaciones de recursos. Cada bug encontrado quedó como test de regresión.

17. ¿Qué no funciona del todo?
El umbral de ME_MIRA (15°) no está validado con la cámara calibrada (en las primeras pruebas casi no se activaba; de él dependen Tímido y Atrevido, que en el video sí funcionaron); la latencia evento → respuesta está estimada (≈ 130–170 ms) pero no medida de punta a punta; el roll vibra girando a la derecha en la zona negativa; Barcelona se confunde con Cerro; la pose queda en 7–10 Hz.

18. ¿Qué muestran el overlay y el log?
El overlay muestra personas con ID, el objetivo, los eventos, el modo, el dueño del cuello y del buzzer, los ángulos comandados y la salud (Hz y edad del cuadro). El log CSV guarda cada transición con marca de tiempo en logs/run_*.csv.

> **Cómo practicar**
> Cada integrante elige 5 preguntas al azar y las responde en voz alta sin mirar; después, explicar el recorrido de un evento de punta a punta: la cámara ve un saludo → percepción (cap. 9) → evento SALUDO (cap. 10) → propuesta del comportamiento → árbitro (cap. 13) → orden a la ESP32 por serie (cap. 7) → trayectoria con curva S (cap. 6) → cabeceos y R2D2.